Aucun document autorisé, sans calculatrice, durée : 55 min
Si vous constatez une erreur dans le document, indiquez les suppositions que vous faites pour répondre à la question.
Chaque partie porte sur ses propres données et extraits de code, indépendamment des autres parties.

Nom, prénom et groupe :

.....

Variables, conditions, boucles et fonctions

Contexte : la borne de recharge EV-BR150

Vous intégrez le bureau d'études d'un équipementier qui développe la borne de recharge pour véhicules électriques EV-BR150, destinée aux parkings d'entreprise. Votre mission porte sur le **logiciel embarqué** de supervision de la borne : il mesure, décide, facture et rend compte. Le cahier des charges qui vous est remis est le suivant.

Architecture et puissance. La borne dispose de **deux points de charge** indépendants pouvant fonctionner simultanément. La puissance délivrée à un véhicule va de 3,7 kW à **150 kW** au maximum. Trois modes d'exploitation sont définis selon la puissance demandée par le véhicule : mode *lent* jusqu'à 22 kW inclus, mode *rapide* au-delà de 22 kW et jusqu'à 50 kW inclus, mode *ultra-rapide* au-delà de 50 kW. Toute demande supérieure à 150 kW est hors du domaine de la borne.

Grandeurs électriques et sécurité. Le raccordement est triphasé 400 V ; la tension réseau mesurée doit rester comprise entre **360 V et 440 V** (bornes incluses). Le courant demandé ne doit pas dépasser **63 A**. La température du module de puissance doit rester **strictement inférieure à 65 °C**. La charge ne s'engage que si le câble est **verrouillé** côté borne. Dès qu'une de ces conditions n'est plus respectée, la charge est interrompue.

Tarification. L'énergie est facturée 0,22 €/kWh en mode lent et rapide, et 0,30 €/kWh au-delà de 50 kW. Un tarif heures creuses de 0,15 €/kWh s'applique aux sessions démarrées entre 22 h et 5 h inclus. Chaque point de charge utilisé ajoute **0,50 € de frais de service** par session. Le stationnement au-delà de 120 minutes peut être majoré.

Exploitation. Les durées sont relevées en minutes par l'automate, alors que les tarifs sont exprimés au kWh : les conversions sont à la charge du logiciel. En fin de journée, l'exploitant attend un bilan : nombre de sessions valides, énergie délivrée, recette. Une session est déclarée **anormale** (donc ni comptée, ni facturée) si sa puissance dépasse 150 kW ou si sa durée est nulle ou négative.

Déroulement du sujet. Les parties A à E suivent les étapes de conception de ce logiciel : on modélise d'abord les **grandeurs** (partie A), puis les **règles de sécurité et de mode** (partie B), puis le **traitement d'une série de sessions** (partie C), puis la **modularisation en fonctions** (partie D), et enfin l'**intégration** de l'ensemble (partie E). Chaque partie redonne toutes les données nécessaires : elle se traite **indépendamment** des autres.

Les cadres pointillés [ ] placés dans les programmes sont à compléter directement sur le sujet.

Reprise du code existant. Une première version de ce logiciel a été écrite par un développeur débutant qui vient de quitter l'entreprise. Vous êtes recruté pour **reprendre et fiabiliser** son travail. Tous les extraits de code fournis dans ce sujet (à lire, à compléter ou à corriger) proviennent de cette version : ils peuvent contenir des erreurs. **Ne supposez donc jamais qu'un extrait est correct** : lisez chaque ligne avec attention pour déterminer ce que le programme fait *réellement*, et non ce qu'il était censé faire. Lorsqu'il vous est demandé d'écrire du code vous-même, appliquez la même exigence : c'est votre code qui servira désormais de référence.

PROJET

Pour votre examen, imprimez de préférence les documents compilés à l'aide de auto-multiple-choice.

Partie A — Modéliser les grandeurs de la session de charge

Cette première partie porte sur le choix des types, la déclaration, la division entière et la priorité des opérateurs, appliqués aux variables qui représenteront les grandeurs de la session de charge. On rappelle que la puissance est exprimée en kW, la durée en minutes, le tarif en €/kWh et le montant en €. Les conversions d'unités sont à la charge du logiciel.

A1 — Déclarer des variables

Question 1
Pour chacune des grandeurs ci-dessous manipulées par la borne, cocher **une seule case** : le type le plus adapté pour la variable qui la représente. (`char` et `string` ne sont proposés ici qu'à titre d'options de reconnaissance.)

Grandeur	Type C le plus adapté
Nombre de badges utilisateurs enregistrés dans la borne	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Tension réseau mesurée (ex. 398,6 V)	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Nombre de coupures réseau détectées depuis la mise en service	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Rendement du convertisseur de puissance (ex. 0,94)	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Un caractère représentant la catégorie d'un utilisateur (ex. 'A', 'B', 'C')	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Index du compteur d'énergie depuis l'installation (ex. 18 452,7 kWh)	◯ `int` ◯ `float` ◯ `char` ◯ `string`

Question 2
Quelle ligne déclare correctement une variable entière `nb_sessions` initialisée à zéro ?

◯ `(nb_sessions == 0);`
◯ `int nb_sessions == 0;`
◯ `int 0 = nb_sessions;`
◯ `int nb_sessions = 0;`
◯ `nb_sessions int = 0;`
◯ `int nb_sessions == 0;`

A2 — Calcul des heures entamées

Pour le ticket de stationnement, la borne convertit la durée (min) en *heures entamées* et en minutes restantes.

Question 3
Pour `heures_entamees` valant 2 et `reste_min` valant 7, qu'affiche la dernière ligne (l'appel à `printf`) du programme ci-dessous ?

◯ `heures_entamees reste_min`
◯ `%d %d`
◯ `722`
◯ `7 2`
◯ `27`
◯ `2 7`

```c
int duree_min      = 185;
int heures_entamees = (duree_min + 59) / 60;
int reste_min      = duree_min % 60;

printf("%d %d\n", heures_entamees, reste_min);
```

PROJET

Pour votre examen, imprimez de préférence les documents compilés à l'aide de auto-multiple-choice.

Question 4
Qu'affiche ce fragment de code complet (avec `duree_min` égal à 185) ?

◯ `4 65` ◯ `3 5` ◯ `3 55` ◯ `4 5` ◯ `3 65` ◯ `4 55`

A3 — Le dépassement

Le stationnement est majoré de 3 € par heure entamée *au-delà* de 120 minutes, en plus de 50 centimes de frais de service.

Un développeur inexpérimenté écrit :

```
int duree_min   = get_int("Durée en minutes : ");
int depassement = duree_min - 120;
int majoration  = depassement / 60 * 300 + 50;

printf("%d cts\n", majoration);
```

Question 5
Qu'affiche ce fragment de code si on entre une durée de 180 minutes ?

◯ 0 cts ◯ 350 cts ◯ 50 cts ◯ 300 cts ◯ 200 cts ◯ 950 cts

Question 6
Qu'affiche ce fragment de code si on entre une durée de 150 minutes ?

◯ 0 cts ◯ 950 cts ◯ 300 cts ◯ 50 cts ◯ 200 cts ◯ 350 cts

Question 7
Proposer une correction du programme pour qu'il calcule correctement le montant à facturer.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

A4 — Trace d'un calcul de coût

Question 8

Avant d'écrire le logiciel, on vérifie la façon dont la borne manipule ses grandeurs. Le fragment ci-dessous traite une session de 7 kW pendant 150 minutes, facturée 0,22 €/kWh.

Compléter le tableau en donnant la valeur de la variable affectée par chacune des lignes repérées `L1` à `L5`, **en respectant le type de la variable** (une valeur entière s'écrit sans décimale). ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

```
float puissance_kw = 7.0;
int   duree_min    = 150;
float tarif_kwh    = 0.22;
float duree_h;
float energie_kwh;
int   energie_kwh_ent;
float cout_euros;

/* L1 */ duree_h         = duree_min / 60.0;
/* L2 */ energie_kwh     = puissance_kw * duree_h;
/* L3 */ energie_kwh_ent = energie_kwh;
/* L4 */ energie_kwh     = puissance_kw * duree_h;
/* L5 */ cout_euros      = energie_kwh * tarif_kwh +
    0.50;
```

Ligne	Variable	Valeur
L1	`duree_h`	
L2	`energie_kwh`	
L3	`energie_kwh_ent`	
L4	`energie_kwh`	
L5	`cout_euros`	

PROJET

Partie B — Décider : sécurité et mode de charge

Deuxième étape : la borne doit décider. Elle autorise ou refuse la charge selon les conditions d'exploitation, puis choisit le mode de charge adapté à la puissance demandée.

B1 — Lire une condition

La borne surveille la température de son module de puissance avec le code suivant.

Question 9

```c
float temperature_c = 58.5;

if (temperature_c < 65.0)
{
    printf("Module OK\n");
}
else
{
    printf("Surchauffe : charge coupee\n");
}
```

Qu'affiche ce fragment de code ?

- ◯ `Surchauffe : charge coupee`
- ◯ Rien : aucune des deux branches n'est exécutée
- ◯ `Module OK`
- ◯ `Module OK` puis, sur la ligne suivante, `Surchauffe : charge coupee`

B2 — Autoriser la charge

Question 10

La borne n'autorise une charge que si **toutes** les conditions d'exploitation sont réunies :

- tension réseau comprise entre 360 V et 440 V, **bornes incluses** ;
- courant demandé **au plus** égal à 63 A ;
- température du module de puissance **strictement** inférieure à 65 °C.

Écrire la condition manquante **directement dans le cadre pointillé**.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    float tension_v     = get_float("Tension reseau (V)   : ");
    float courant_a     = get_float("Courant demande (A)  : ");
    int   temperature_c = get_int("Temperature module (C) : ");

    if (                                                    )
    {
        printf("Charge autorisee\n");
    }
    else
    {
        printf("Charge refusee\n");
    }

    return 0;
}
```

Justifier brièvement le choix des opérateurs aux bornes de la plage de tension.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

SUJET

B3 — Surveiller la sécurité

Question 11

Un technicien a écrit le module de surveillance ci-dessous. Compilé sans option d'avertissement, il affiche parfois « Charge interrompue » alors qu'aucun défaut n'a été signalé.

```
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    float tension_v = get_float("Tension reseau (V)  : ");
    float courant_a = get_float("Courant demande (A) : ");
    int   mode      = get_int("Mode de charge (0/1/2) : ");
    int   defaut    = 0;

    if (360.0 <= tension_v <= 440.0)
    {
        printf("Tension hors plage\n");
        defaut = 1;
    }

    if (courant_a > 63.0 && mode = 2)
    {
        printf("Surintensite en mode ultra-rapide\n");
        defaut = 1;
    }

    if (defaut == 1);
    {
        printf("Charge interrompue\n");
    }

    return 0;
}
```

Repérer la ou les erreurs présentes dans ce programme en les **numérotant directement sur le listing** (1, 2…). Puis, dans le cadre ci-dessous, donner pour chaque numéro la **ligne corrigée** et le comportement fautif que la correction supprime.

PRO

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Partie C — Répéter : traiter une série de sessions

Troisième étape : la borne répète sans cesse des traitements. On étudie ici l'attente du verrouillage du câble, le cumul de l'énergie des sessions de la journée, la simulation d'une recharge à puissance dégressive, puis une montée en charge par paliers.

C1 — Attente du verrouillage

Question 12
Au branchement, la borne revérifie le verrouillage du câble toutes les 15 secondes tant que la durée d'attente écoulée ne dépasse pas 60 secondes.

```
int temps_s   = 0;
int nb_essais = 0;

while (temps_s < 60)
{
    temps_s = temps_s + 15;
    nb_essais++;
}

printf("%d %d\n", nb_essais, temps_s);
```

Qu'affiche ce fragment ?

◯ 4 45 ◯ 5 60 ◯ 4 60 ◯ 5 75 ◯ 4 75 ◯ 3 45

PROJET

C2 — Cumul de l'énergie

Question 13

Pour le bilan d'un point de charge, un développeur inexpérimenté a écrit le programme suivant, qui cumule l'énergie délivrée par chacune des sessions de la journée.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int   nb_sessions    = get_int("Nombre de sessions : ");
    float energie_totale = 0.0;
    int   i              = 0;

    while (i < nb_sessions)
    {
        float energie_kwh = get_float("Energie de la session (kWh) : ");
        energie_totale = energie_totale + energie_kwh;
    }

    printf("Energie totale : %.1f kWh\n", energie_totale);

    return 0;
}
```

Ce programme ne se comporte pas comme prévu. Décrire le comportement observé à l'exécution, puis corriger le programme **directement sur le code**. Justifier ci-dessous.

PROJET

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

C3 — Recharge à puissance dégressive

Pour préserver la batterie, la borne réduit la puissance délivrée lorsque le niveau de charge augmente : le niveau gagne 20 points par pas de 10 minutes sous 60 %, puis 10 points de 60 à 80 %, puis 5 points au-delà. Le fragment ci-dessous simule cette recharge (`gain_pct` est le nombre de points de pourcentage gagnés pendant un pas de 10 minutes).

Question 14

Compléter le tableau en donnant, à la fin de **chaque tour de boucle**, les valeurs de `gain_pct`, `niveau_pct` et `minutes`. Numéroter vous-même les tours suivants et **s'arrêter au dernier tour effectivement exécuté** (les lignes en trop restent vides). ..... ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

```
int niveau_pct = 40;
int gain_pct   = 20;
int minutes    = 0;

while (niveau_pct < 90)
{
    if (niveau_pct >= 80)
    {
        gain_pct = 5;
    }
    else if (niveau_pct >= 60)
    {
        gain_pct = 10;
    }

    niveau_pct = niveau_pct + gain_pct;
    minutes    = minutes + 10;
}

printf("%d %d\n", minutes, niveau_pct);
```

Tour	gain_pct	niveau_pct	minutes
1			
2			

Question 15

Qu'affiche ce programme après exécution complète de la boucle ?

Note au correcteur : évaluer en cohérence avec le tableau rempli par l'étudiant à la question précédente (la bonne réponse est celle qui découle de sa *trace, même erronée), afin de ne pas pénaliser deux fois la même erreur.*

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

C4 — Montée en charge par paliers

La borne monte le niveau de charge par paliers de 7 points de pourcentage, en partant de 15 %. Le cahier des charges impose que :

- la boucle s'arrête dès que le niveau atteint **ou dépasse** le niveau cible saisi par l'utilisateur ;
- le niveau affiché ne dépasse **jamais** 100 %.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int niveau_pct   = 15;
    int nb_paliers   = 0;
    int niveau_cible = get_int("Niveau de charge souhaite (%) : ");

    while (                                                    )
    {
        niveau_pct =                                                ;

        if (                                                    )
        {
            niveau_pct = 100;
        }

        nb_paliers++;
        printf("Palier %d : %d %%\n", nb_paliers, niveau_pct);
    }

    printf("Charge terminee en %d paliers.\n", nb_paliers);

    return 0;
}
```

Question 16

Compléter les trois éléments manquants **directement dans les cadres pointillés**.

. . . . . ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Question 17

Indiquer ce que le programme affiche si l'utilisateur saisit un niveau cible de 40 %.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PRO

Partie D — Modulariser : des fonctions pour la borne

Quatrième étape : le logiciel grossit. On isole les traitements (comptage, facturation, décision d'engagement) dans des fonctions réutilisables et testables séparément.

D1 — Appeler une fonction existante

Le calcul du montant d'une session est confié à la fonction `cout_session`. On considère que cette fonction est **déjà écrite et implémentée dans le programme** : il n'est pas utile de connaître son contenu, seul son prototype compte :

```
float cout_session(float puissance_kw, float duree_h, float tarif_kwh);
```

Quatre appels à cette fonction apparaissent dans le programme :

```
/* Prototype disponible :
   float cout_session(float puissance_kw, float duree_h, float tarif_kwh); */

float c1 = cout_session(22.0, 1.5, 0.22);   /* appel 1 */
float c2 = cout_session(0.22, 22.0, 1.5);   /* appel 2 */
float c3 = cout_session(22.0, 1.5);         /* appel 3 */
float c4 = cout_session(22.0, 1.5, 0.22, 0.50); /* appel 4 */
```

Question 18 ♣
Cocher **tous** les appels acceptés par le compilateur.

◯ appel 1 ◯ appel 2 ◯ appel 3 ◯ appel 4

Question 19
Parmi les appels de la question précédent, indiquer lequel est la **bonne version** pour calculer le coût d'une session de 22 kW pendant 1,5 h à 0,22 €/kWh ?

◯ appel 1 ◯ appel 2 ◯ appel 3 ◯ appel 4

PROJET

D2 — Portée d'une variable

Un développeur tente de compter les sessions de la borne avec le programme suivant :

```c
void enregistrer_session(void)
{
    int compteur_total = 0;
    compteur_total++;
    printf("%d ", compteur_total);
}

int main(void)
{
    int compteur_total = 0;

    for (int i = 0; i < 3; i++)
    {
        enregistrer_session();
    }

    printf("| %d\n", compteur_total);

    return 0;
}
```

Question 20
Qu'affiche ce programme ?

◯ `1 1 1 | 0` ◯ `1 2 3 | 0` ◯ `1 2 3 | 3` ◯ `1 1 1 | 3`

Question 21
Réécrire la fonction `enregistrer_session` pour qu'elle **reçoive le compteur en paramètre** et **retourne sa nouvelle valeur** (sans variable globale), puis écrire la boucle de `main` qui l'utilise, de sorte que le programme affiche `1 2 3 | 3`.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

D3 — Écrire une fonction de décision

Question 22

On modularise maintenant la décision d'engagement de la charge.

On souhaite la fonction `int determiner_mode(float puissance_kw, int cable_verrouille)` qui retourne un **code entier** caractérisant la décision de la borne :

- si le câble n'est pas verrouillé (`cable_verrouille` vaut 0), retourner `-1` quelle que soit la puissance ;
- sinon, retourner `0` si la puissance demandée ne dépasse pas 22 kW, `1` si elle est comprise entre 22 kW (exclu) et 50 kW (inclus), `2` si elle dépasse 50 kW sans dépasser 150 kW ;
- si la puissance demandée dépasse 150 kW, retourner `3` (demande hors domaine de la borne).

La fonction ne doit rien afficher et doit retourner une valeur dans **tous** les cas.

Ecrire la définition (son corps) de cette fonction.

PROJET

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Partie E — Intégrer : le bilan de journée

Dernière étape : on assemble une fonction déjà écrite, une boucle et une structure de choix pour traiter les demandes de la journée et compter les sessions refusées.

Question 23

On dispose de la fonction `int determiner_mode(float puissance_kw, int cable_verrouille)`, que l'on considère **correcte**. Elle retourne `-1` si le câble n'est pas verrouillé, `0` en mode lent, `1` en mode rapide, `2` en mode ultra-rapide et `3` si la demande dépasse 150 kW. Sa définition est placée **après** le `main` ; son prototype est rappelé en tête du fichier.

Compléter les cadres pointillés pour que le programme traite les `nb_sessions` demandes de la journée, compte les sessions refusées (câble non verrouillé ou demande hors domaine) et affiche le mode des autres. Le choix selon le code retourné est réalisé avec une structure `switch`.

```c
#include <stdio.h>
#include <cs50.h>

int determiner_mode(float puissance_kw, int cable_verrouille);

int main(void)
{
    int nb_sessions = get_int("Nombre de sessions : ");
    int nb_refus    = 0;

    for (int i = 1; .....; i++)
    {
        float puissance_kw     = get_float("Puissance demandee (kW) : ");
        int   cable_verrouille = get_int("Cable verrouille (1/0) : ");

        int code = .....;

        switch (.....)
        {
            case .....:
            case 3:
                nb_refus++;
                break;
            case .....:
                printf("Session %d : mode lent\n", i);
                break;
            .....
                printf("Session %d : mode rapide ou ultra-rapide\n", i);
                break;
        }
    }

    printf("Sessions refusees : %d\n", nb_refus);

    return 0;
}

/* La definition de determiner_mode se trouve ici (supposee correcte). */
```

..... ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Nom, prénom et groupe :

.....

Variables, conditions, boucles et fonctions

Contexte : la borne de recharge EV-BR150

Vous intégrez le bureau d'études d'un équipementier qui développe la borne de recharge pour véhicules électriques EV-BR150, destinée aux parkings d'entreprise. Votre mission porte sur le **logiciel embarqué** de supervision de la borne : il mesure, décide, facture et rend compte. Le cahier des charges qui vous est remis est le suivant.

Architecture et puissance. La borne dispose de **deux points de charge** indépendants pouvant fonctionner simultanément. La puissance délivrée à un véhicule va de 3,7 kW à **150 kW** au maximum. Trois modes d'exploitation sont définis selon la puissance demandée par le véhicule : mode *lent* jusqu'à 22 kW inclus, mode *rapide* au-delà de 22 kW et jusqu'à 50 kW inclus, mode *ultra-rapide* au-delà de 50 kW. Toute demande supérieure à 150 kW est hors du domaine de la borne.

Grandeurs électriques et sécurité. Le raccordement est triphasé 400 V ; la tension réseau mesurée doit rester comprise entre **360 V et 440 V** (bornes incluses). Le courant demandé ne doit pas dépasser **63 A**. La température du module de puissance doit rester **strictement inférieure à 65 °C**. La charge ne s'engage que si le câble est **verrouillé** côté borne. Dès qu'une de ces conditions n'est plus respectée, la charge est interrompue.

Tarification. L'énergie est facturée 0,22 €/kWh en mode lent et rapide, et 0,30 €/kWh au-delà de 50 kW. Un tarif heures creuses de 0,15 €/kWh s'applique aux sessions démarrées entre 22 h et 5 h inclus. Chaque point de charge utilisé ajoute **0,50 € de frais de service** par session. Le stationnement au-delà de 120 minutes peut être majoré.

Exploitation. Les durées sont relevées en minutes par l'automate, alors que les tarifs sont exprimés au kWh : les conversions sont à la charge du logiciel. En fin de journée, l'exploitant attend un bilan : nombre de sessions valides, énergie délivrée, recette. Une session est déclarée **anormale** (donc ni comptée, ni facturée) si sa puissance dépasse 150 kW ou si sa durée est nulle ou négative.

Déroulement du sujet. Les parties A à E suivent les étapes de conception de ce logiciel : on modélise d'abord les **grandeurs** (partie A), puis les **règles de sécurité et de mode** (partie B), puis le **traitement d'une série de sessions** (partie C), puis la **modularisation en fonctions** (partie D), et enfin l'**intégration** de l'ensemble (partie E). Chaque partie redonne toutes les données nécessaires : elle se traite **indépendamment** des autres.

Les cadres pointillés [ ] placés dans les programmes sont à compléter directement sur le sujet.

Reprise du code existant. Une première version de ce logiciel a été écrite par un développeur débutant qui vient de quitter l'entreprise. Vous êtes recruté pour **reprendre et fiabiliser** son travail. Tous les extraits de code fournis dans ce sujet (à lire, à compléter ou à corriger) proviennent de cette version : ils peuvent contenir des erreurs. **Ne supposez donc jamais qu'un extrait est correct** : lisez chaque ligne avec attention pour déterminer ce que le programme fait *réellement*, et non ce qu'il était censé faire. Lorsqu'il vous est demandé d'écrire du code vous-même, appliquez la même exigence : c'est votre code qui servira désormais de référence.

PROJET

Pour votre examen, imprimez de préférence les documents compilés à l'aide de auto-multiple-choice.

Partie A — Modéliser les grandeurs de la session de charge

Cette première partie porte sur le choix des types, la déclaration, la division entière et la priorité des opérateurs, appliqués aux variables qui représenteront les grandeurs de la session de charge. On rappelle que la puissance est exprimée en kW, la durée en minutes, le tarif en €/kWh et le montant en €. Les conversions d'unités sont à la charge du logiciel.

A1 — Déclarer des variables

Question 1
Pour chacune des grandeurs ci-dessous manipulées par la borne, cocher **une seule case** : le type le plus adapté pour la variable qui la représente. (`char` et `string` ne sont proposés ici qu'à titre d'options de reconnaissance.)

Grandeur	Type C le plus adapté
Nombre de badges utilisateurs enregistrés dans la borne	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Tension réseau mesurée (ex. 398,6 V)	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Nombre de coupures réseau détectées depuis la mise en service	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Rendement du convertisseur de puissance (ex. 0,94)	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Un caractère représentant la catégorie d'un utilisateur (ex. 'A', 'B', 'C')	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Index du compteur d'énergie depuis l'installation (ex. 18 452,7 kWh)	◯ `int` ◯ `float` ◯ `char` ◯ `string`

Question 2
Quelle ligne déclare correctement une variable entière `nb_sessions` initialisée à zéro ?

◯ `int nb_sessions == 0;`
◯ `nb_sessions int = 0;`

◯ `int nb_sessions = 0;`
◯ `int 0 = nb_sessions;`

◯ `int nb_sessions == 0;`
◯ `(nb_sessions == 0);`

A2 — Calcul des heures entamées

Pour le ticket de stationnement, la borne convertit la durée (min) en *heures entamées* et en minutes restantes.

Question 3
Pour `heures_entamees` valant 2 et `reste_min` valant 7, qu'affiche la dernière ligne (l'appel à `printf`) du programme ci-dessous ?

◯ `722`
◯ `heures_entamees reste_min`

◯ `7 2`
◯ `2 7`
◯ `%d %d`

◯ `27`

```c
int duree_min      = 185;
int heures_entamees = (duree_min + 59) / 60;
int reste_min      = duree_min % 60;

printf("%d %d\n", heures_entamees, reste_min);
```

PROJET

Pour votre examen, imprimez de préférence les documents compilés à l'aide de auto-multiple-choice.

Question 4
Qu'affiche ce fragment de code complet (avec `duree_min` égal à 185) ?

◯ `4 55` ◯ `4 5` ◯ `3 5` ◯ `3 55` ◯ `3 65` ◯ `4 65`

A3 — Le dépassement

Le stationnement est majoré de 3 € par heure entamée *au-delà* de 120 minutes, en plus de 50 centimes de frais de service.

Un développeur inexpérimenté écrit :

```c
int duree_min   = get_int("Durée en minutes : ");
int depassement = duree_min - 120;
int majoration  = depassement / 60 * 300 + 50;

printf("%d cts\n", majoration);
```

Question 5
Qu'affiche ce fragment de code si on entre une durée de 180 minutes ?

◯ 350 cts ◯ 50 cts ◯ 300 cts ◯ 200 cts ◯ 0 cts ◯ 950 cts

Question 6
Qu'affiche ce fragment de code si on entre une durée de 150 minutes ?

◯ 0 cts ◯ 300 cts ◯ 350 cts ◯ 950 cts ◯ 50 cts ◯ 200 cts

Question 7
Proposer une correction du programme pour qu'il calcule correctement le montant à facturer.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

A4 — Trace d'un calcul de coût

Question 8

Avant d'écrire le logiciel, on vérifie la façon dont la borne manipule ses grandeurs. Le fragment ci-dessous traite une session de 7 kW pendant 150 minutes, facturée 0,22 €/kWh.

Compléter le tableau en donnant la valeur de la variable affectée par chacune des lignes repérées `L1` à `L5`, **en respectant le type de la variable** (une valeur entière s'écrit sans décimale). ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

```
float puissance_kw = 7.0;
int   duree_min    = 150;
float tarif_kwh    = 0.22;
float duree_h;
float energie_kwh;
int   energie_kwh_ent;
float cout_euros;

/* L1 */ duree_h         = duree_min / 60.0;
/* L2 */ energie_kwh     = puissance_kw * duree_h;
/* L3 */ energie_kwh_ent = energie_kwh;
/* L4 */ energie_kwh     = puissance_kw * duree_h;
/* L5 */ cout_euros      = energie_kwh * tarif_kwh +
    0.50;
```

Ligne	Variable	Valeur
L1	`duree_h`	
L2	`energie_kwh`	
L3	`energie_kwh_ent`	
L4	`energie_kwh`	
L5	`cout_euros`	

PROJET

Partie B — Décider : sécurité et mode de charge

Deuxième étape : la borne doit décider. Elle autorise ou refuse la charge selon les conditions d'exploitation, puis choisit le mode de charge adapté à la puissance demandée.

B1 — Lire une condition

La borne surveille la température de son module de puissance avec le code suivant.

Question 9

```c
float temperature_c = 58.5;

if (temperature_c < 65.0)
{
    printf("Module OK\n");
}
else
{
    printf("Surchauffe : charge coupee\n");
}
```

Qu'affiche ce fragment de code ?

- ◯ `Module OK` puis, sur la ligne suivante, `Surchauffe : charge coupee`
- ◯ `Module OK`
- ◯ Rien : aucune des deux branches n'est exécutée
- ◯ `Surchauffe : charge coupee`

B2 — Autoriser la charge

Question 10

La borne n'autorise une charge que si **toutes** les conditions d'exploitation sont réunies :

- tension réseau comprise entre 360 V et 440 V, **bornes incluses** ;
- courant demandé **au plus** égal à 63 A ;
- température du module de puissance **strictement** inférieure à 65 °C.

Écrire la condition manquante **directement dans le cadre pointillé**.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    float tension_v     = get_float("Tension reseau (V)   : ");
    float courant_a     = get_float("Courant demande (A)  : ");
    int   temperature_c = get_int("Temperature module (C) : ");

    if (                                                    )
    {
        printf("Charge autorisee\n");
    }
    else
    {
        printf("Charge refusee\n");
    }

    return 0;
}
```

Justifier brièvement le choix des opérateurs aux bornes de la plage de tension.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

SUJET

B3 — Surveiller la sécurité

Question 11

Un technicien a écrit le module de surveillance ci-dessous. Compilé sans option d'avertissement, il affiche parfois « Charge interrompue » alors qu'aucun défaut n'a été signalé.

```
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    float tension_v = get_float("Tension reseau (V)  : ");
    float courant_a = get_float("Courant demande (A) : ");
    int   mode      = get_int("Mode de charge (0/1/2) : ");
    int   defaut    = 0;

    if (360.0 <= tension_v <= 440.0)
    {
        printf("Tension hors plage\n");
        defaut = 1;
    }

    if (courant_a > 63.0 && mode = 2)
    {
        printf("Surintensite en mode ultra-rapide\n");
        defaut = 1;
    }

    if (defaut == 1);
    {
        printf("Charge interrompue\n");
    }

    return 0;
}
```

Repérer la ou les erreurs présentes dans ce programme en les **numérotant directement sur le listing** (1, 2…). Puis, dans le cadre ci-dessous, donner pour chaque numéro la **ligne corrigée** et le comportement fautif que la correction supprime.

PRO

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Partie C — Répéter : traiter une série de sessions

Troisième étape : la borne répète sans cesse des traitements. On étudie ici l'attente du verrouillage du câble, le cumul de l'énergie des sessions de la journée, la simulation d'une recharge à puissance dégressive, puis une montée en charge par paliers.

C1 — Attente du verrouillage

Question 12
Au branchement, la borne revérifie le verrouillage du câble toutes les 15 secondes tant que la durée d'attente écoulée ne dépasse pas 60 secondes.

```c
int temps_s   = 0;
int nb_essais = 0;

while (temps_s < 60)
{
    temps_s = temps_s + 15;
    nb_essais++;
}

printf("%d %d\n", nb_essais, temps_s);
```

Qu'affiche ce fragment ?

◯ 4 45 ◯ 3 45 ◯ 4 75 ◯ 5 60 ◯ 4 60 ◯ 5 75

PROJET

C2 — Cumul de l'énergie

Question 13

Pour le bilan d'un point de charge, un développeur inexpérimenté a écrit le programme suivant, qui cumule l'énergie délivrée par chacune des sessions de la journée.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int   nb_sessions    = get_int("Nombre de sessions : ");
    float energie_totale = 0.0;
    int   i              = 0;

    while (i < nb_sessions)
    {
        float energie_kwh = get_float("Energie de la session (kWh) : ");
        energie_totale = energie_totale + energie_kwh;
    }

    printf("Energie totale : %.1f kWh\n", energie_totale);

    return 0;
}
```

Ce programme ne se comporte pas comme prévu. Décrire le comportement observé à l'exécution, puis corriger le programme **directement sur le code**. Justifier ci-dessous.

PROJET

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

C3 — Recharge à puissance dégressive

Pour préserver la batterie, la borne réduit la puissance délivrée lorsque le niveau de charge augmente : le niveau gagne 20 points par pas de 10 minutes sous 60 %, puis 10 points de 60 à 80 %, puis 5 points au-delà. Le fragment ci-dessous simule cette recharge (`gain_pct` est le nombre de points de pourcentage gagnés pendant un pas de 10 minutes).

Question 14

Compléter le tableau en donnant, à la fin de **chaque tour de boucle**, les valeurs de `gain_pct`, `niveau_pct` et `minutes`. Numéroter vous-même les tours suivants et **s'arrêter au dernier tour effectivement exécuté** (les lignes en trop restent vides). ..... ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

```
int niveau_pct = 40;
int gain_pct   = 20;
int minutes    = 0;

while (niveau_pct < 90)
{
    if (niveau_pct >= 80)
    {
        gain_pct = 5;
    }
    else if (niveau_pct >= 60)
    {
        gain_pct = 10;
    }

    niveau_pct = niveau_pct + gain_pct;
    minutes    = minutes + 10;
}

printf("%d %d\n", minutes, niveau_pct);
```

Tour	gain_pct	niveau_pct	minutes
1			
2			

Question 15

Qu'affiche ce programme après exécution complète de la boucle ?

Note au correcteur : évaluer en cohérence avec le tableau rempli par l'étudiant à la question précédente (la bonne réponse est celle qui découle de sa *trace, même erronée), afin de ne pas pénaliser deux fois la même erreur.*

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

C4 — Montée en charge par paliers

La borne monte le niveau de charge par paliers de 7 points de pourcentage, en partant de 15 %. Le cahier des charges impose que :

- la boucle s'arrête dès que le niveau atteint **ou dépasse** le niveau cible saisi par l'utilisateur ;
- le niveau affiché ne dépasse **jamais** 100 %.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int niveau_pct   = 15;
    int nb_paliers   = 0;
    int niveau_cible = get_int("Niveau de charge souhaite (%) : ");

    while (                                                    )
    {
        niveau_pct =                                                    ;

        if (                                                    )
        {
            niveau_pct = 100;
        }

        nb_paliers++;
        printf("Palier %d : %d %%\n", nb_paliers, niveau_pct);
    }

    printf("Charge terminee en %d paliers.\n", nb_paliers);

    return 0;
}
```

Question 16

Compléter les trois éléments manquants **directement dans les cadres pointillés**.

. . . . . ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Question 17

Indiquer ce que le programme affiche si l'utilisateur saisit un niveau cible de 40 %.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROG

Partie D — Modulariser : des fonctions pour la borne

Quatrième étape : le logiciel grossit. On isole les traitements (comptage, facturation, décision d'engagement) dans des fonctions réutilisables et testables séparément.

D1 — Appeler une fonction existante

Le calcul du montant d'une session est confié à la fonction `cout_session`. On considère que cette fonction est **déjà écrite et implémentée dans le programme** : il n'est pas utile de connaître son contenu, seul son prototype compte :

```
float cout_session(float puissance_kw, float duree_h, float tarif_kwh);
```

Quatre appels à cette fonction apparaissent dans le programme :

```
/* Prototype disponible :
   float cout_session(float puissance_kw, float duree_h, float tarif_kwh); */

float c1 = cout_session(22.0, 1.5, 0.22);   /* appel 1 */
float c2 = cout_session(0.22, 22.0, 1.5);   /* appel 2 */
float c3 = cout_session(22.0, 1.5);         /* appel 3 */
float c4 = cout_session(22.0, 1.5, 0.22, 0.50); /* appel 4 */
```

Question 18 ♣
Cocher **tous** les appels acceptés par le compilateur.

◯ appel 1 ◯ appel 2 ◯ appel 3 ◯ appel 4

Question 19
Parmi les appels de la question précédent, indiquer lequel est la **bonne version** pour calculer le coût d'une session de 22 kW pendant 1,5 h à 0,22 €/kWh ?

◯ appel 1 ◯ appel 2 ◯ appel 3 ◯ appel 4

PROJET

D2 — Portée d'une variable

Un développeur tente de compter les sessions de la borne avec le programme suivant :

```c
void enregistrer_session(void)
{
    int compteur_total = 0;
    compteur_total++;
    printf("%d ", compteur_total);
}

int main(void)
{
    int compteur_total = 0;

    for (int i = 0; i < 3; i++)
    {
        enregistrer_session();
    }

    printf("| %d\n", compteur_total);

    return 0;
}
```

Question 20
Qu'affiche ce programme ?

◯ `1 1 1 | 3` ◯ `1 2 3 | 0` ◯ `1 2 3 | 3` ◯ `1 1 1 | 0`

Question 21
Réécrire la fonction `enregistrer_session` pour qu'elle **reçoive le compteur en paramètre** et **retourne sa nouvelle valeur** (sans variable globale), puis écrire la boucle de `main` qui l'utilise, de sorte que le programme affiche `1 2 3 | 3`.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

D3 — Écrire une fonction de décision

Question 22

On modularise maintenant la décision d'engagement de la charge.

On souhaite la fonction `int determiner_mode(float puissance_kw, int cable_verrouille)` qui retourne un **code entier** caractérisant la décision de la borne :

- si le câble n'est pas verrouillé (`cable_verrouille` vaut 0), retourner `-1` quelle que soit la puissance ;
- sinon, retourner `0` si la puissance demandée ne dépasse pas 22 kW, `1` si elle est comprise entre 22 kW (exclu) et 50 kW (inclus), `2` si elle dépasse 50 kW sans dépasser 150 kW ;
- si la puissance demandée dépasse 150 kW, retourner `3` (demande hors domaine de la borne).

La fonction ne doit rien afficher et doit retourner une valeur dans **tous** les cas.

Ecrire la définition (son corps) de cette fonction.

PROJET

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Partie E — Intégrer : le bilan de journée

Dernière étape : on assemble une fonction déjà écrite, une boucle et une structure de choix pour traiter les demandes de la journée et compter les sessions refusées.

Question 23

On dispose de la fonction `int determiner_mode(float puissance_kw, int cable_verrouille)`, que l'on considère **correcte**. Elle retourne `-1` si le câble n'est pas verrouillé, `0` en mode lent, `1` en mode rapide, `2` en mode ultra-rapide et `3` si la demande dépasse 150 kW. Sa définition est placée **après** le `main` ; son prototype est rappelé en tête du fichier.

Compléter les cadres pointillés pour que le programme traite les `nb_sessions` demandes de la journée, compte les sessions refusées (câble non verrouillé ou demande hors domaine) et affiche le mode des autres. Le choix selon le code retourné est réalisé avec une structure `switch`.

```c
#include <stdio.h>
#include <cs50.h>

int determiner_mode(float puissance_kw, int cable_verrouille);

int main(void)
{
    int nb_sessions = get_int("Nombre de sessions : ");
    int nb_refus    = 0;

    for (int i = 1; .....; i++)
    {
        float puissance_kw     = get_float("Puissance demandee (kW) : ");
        int   cable_verrouille = get_int("Cable verrouille (1/0) : ");

        int code = .....;

        switch (.....)
        {
            case .....:
            case 3:
                nb_refus++;
                break;
            case .....:
                printf("Session %d : mode lent\n", i);
                break;
            .....
                printf("Session %d : mode rapide ou ultra-rapide\n", i);
                break;
        }
    }

    printf("Sessions refusees : %d\n", nb_refus);

    return 0;
}

/* La definition de determiner_mode se trouve ici (supposee correcte). */
```

..... ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Nom, prénom et groupe :

.....

Variables, conditions, boucles et fonctions

Contexte : la borne de recharge EV-BR150

Vous intégrez le bureau d'études d'un équipementier qui développe la borne de recharge pour véhicules électriques EV-BR150, destinée aux parkings d'entreprise. Votre mission porte sur le **logiciel embarqué** de supervision de la borne : il mesure, décide, facture et rend compte. Le cahier des charges qui vous est remis est le suivant.

Architecture et puissance. La borne dispose de **deux points de charge** indépendants pouvant fonctionner simultanément. La puissance délivrée à un véhicule va de 3,7 kW à **150 kW** au maximum. Trois modes d'exploitation sont définis selon la puissance demandée par le véhicule : mode *lent* jusqu'à 22 kW inclus, mode *rapide* au-delà de 22 kW et jusqu'à 50 kW inclus, mode *ultra-rapide* au-delà de 50 kW. Toute demande supérieure à 150 kW est hors du domaine de la borne.

Grandeurs électriques et sécurité. Le raccordement est triphasé 400 V ; la tension réseau mesurée doit rester comprise entre **360 V et 440 V** (bornes incluses). Le courant demandé ne doit pas dépasser **63 A**. La température du module de puissance doit rester **strictement inférieure à 65 °C**. La charge ne s'engage que si le câble est **verrouillé** côté borne. Dès qu'une de ces conditions n'est plus respectée, la charge est interrompue.

Tarification. L'énergie est facturée 0,22 €/kWh en mode lent et rapide, et 0,30 €/kWh au-delà de 50 kW. Un tarif heures creuses de 0,15 €/kWh s'applique aux sessions démarrées entre 22 h et 5 h inclus. Chaque point de charge utilisé ajoute **0,50 € de frais de service** par session. Le stationnement au-delà de 120 minutes peut être majoré.

Exploitation. Les durées sont relevées en minutes par l'automate, alors que les tarifs sont exprimés au kWh : les conversions sont à la charge du logiciel. En fin de journée, l'exploitant attend un bilan : nombre de sessions valides, énergie délivrée, recette. Une session est déclarée **anormale** (donc ni comptée, ni facturée) si sa puissance dépasse 150 kW ou si sa durée est nulle ou négative.

Déroulement du sujet. Les parties A à E suivent les étapes de conception de ce logiciel : on modélise d'abord les **grandeurs** (partie A), puis les **règles de sécurité et de mode** (partie B), puis le **traitement d'une série de sessions** (partie C), puis la **modularisation en fonctions** (partie D), et enfin l'**intégration** de l'ensemble (partie E). Chaque partie redonne toutes les données nécessaires : elle se traite **indépendamment** des autres.

Les cadres pointillés placés dans les programmes sont à compléter directement sur le sujet.

Reprise du code existant. Une première version de ce logiciel a été écrite par un développeur débutant qui vient de quitter l'entreprise. Vous êtes recruté pour **reprendre et fiabiliser** son travail. Tous les extraits de code fournis dans ce sujet (à lire, à compléter ou à corriger) proviennent de cette version : ils peuvent contenir des erreurs. **Ne supposez donc jamais qu'un extrait est correct** : lisez chaque ligne avec attention pour déterminer ce que le programme fait *réellement*, et non ce qu'il était censé faire. Lorsqu'il vous est demandé d'écrire du code vous-même, appliquez la même exigence : c'est votre code qui servira désormais de référence.

PROJET

Pour votre examen, imprimez de préférence les documents compilés à l'aide de auto-multiple-choice.

Partie A — Modéliser les grandeurs de la session de charge

Cette première partie porte sur le choix des types, la déclaration, la division entière et la priorité des opérateurs, appliqués aux variables qui représenteront les grandeurs de la session de charge. On rappelle que la puissance est exprimée en kW, la durée en minutes, le tarif en €/kWh et le montant en €. Les conversions d'unités sont à la charge du logiciel.

A1 — Déclarer des variables

Question 1
Pour chacune des grandeurs ci-dessous manipulées par la borne, cocher **une seule case** : le type le plus adapté pour la variable qui la représente. (`char` et `string` ne sont proposés ici qu'à titre d'options de reconnaissance.)

Grandeur	Type C le plus adapté
Nombre de badges utilisateurs enregistrés dans la borne	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Tension réseau mesurée (ex. 398,6 V)	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Nombre de coupures réseau détectées depuis la mise en service	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Rendement du convertisseur de puissance (ex. 0,94)	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Un caractère représentant la catégorie d'un utilisateur (ex. 'A', 'B', 'C')	◯ `int` ◯ `float` ◯ `char` ◯ `string`
Index du compteur d'énergie depuis l'installation (ex. 18 452,7 kWh)	◯ `int` ◯ `float` ◯ `char` ◯ `string`

Question 2
Quelle ligne déclare correctement une variable entière `nb_sessions` initialisée à zéro ?

◯ `(nb_sessions == 0);`
◯ `nb_sessions int = 0;`
◯ `int nb_sessions == 0;`
◯ `int 0 = nb_sessions;`
◯ `int nb_sessions = 0;`
◯ `int nb_sessions == 0;`

A2 — Calcul des heures entamées

Pour le ticket de stationnement, la borne convertit la durée (min) en *heures entamées* et en minutes restantes.

Question 3
Pour `heures_entamees` valant 2 et `reste_min` valant 7, qu'affiche la dernière ligne (l'appel à `printf`) du programme ci-dessous ?

◯ `722`
◯ `heures_entamees reste_min`
◯ `%d %d`
◯ `7 2`
◯ `27`
◯ `2 7`

```c
int duree_min      = 185;
int heures_entamees = (duree_min + 59) / 60;
int reste_min      = duree_min % 60;

printf("%d %d\n", heures_entamees, reste_min);
```

PROJET

Pour votre examen, imprimez de préférence les documents compilés à l'aide de auto-multiple-choice.

Question 4
Qu'affiche ce fragment de code complet (avec `duree_min` égal à 185) ?

◯ `3 5` ◯ `3 65` ◯ `3 55` ◯ `4 65` ◯ `4 55` ◯ `4 5`

A3 — Le dépassement

Le stationnement est majoré de 3 € par heure entamée *au-delà* de 120 minutes, en plus de 50 centimes de frais de service.

Un développeur inexpérimenté écrit :

```
int duree_min   = get_int("Durée en minutes : ");
int depassement = duree_min - 120;
int majoration  = depassement / 60 * 300 + 50;

printf("%d cts\n", majoration);
```

Question 5
Qu'affiche ce fragment de code si on entre une durée de 180 minutes ?

◯ 350 cts ◯ 0 cts ◯ 200 cts ◯ 950 cts ◯ 50 cts ◯ 300 cts

Question 6
Qu'affiche ce fragment de code si on entre une durée de 150 minutes ?

◯ 200 cts ◯ 50 cts ◯ 0 cts ◯ 300 cts ◯ 950 cts ◯ 350 cts

Question 7
Proposer une correction du programme pour qu'il calcule correctement le montant à facturer.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

A4 — Trace d'un calcul de coût

Question 8

Avant d'écrire le logiciel, on vérifie la façon dont la borne manipule ses grandeurs. Le fragment ci-dessous traite une session de 7 kW pendant 150 minutes, facturée 0,22 €/kWh.

Compléter le tableau en donnant la valeur de la variable affectée par chacune des lignes repérées `L1` à `L5`, **en respectant le type de la variable** (une valeur entière s'écrit sans décimale). ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

```
float puissance_kw = 7.0;
int   duree_min    = 150;
float tarif_kwh    = 0.22;
float duree_h;
float energie_kwh;
int   energie_kwh_ent;
float cout_euros;

/* L1 */ duree_h         = duree_min / 60.0;
/* L2 */ energie_kwh     = puissance_kw * duree_h;
/* L3 */ energie_kwh_ent = energie_kwh;
/* L4 */ energie_kwh     = puissance_kw * duree_h;
/* L5 */ cout_euros      = energie_kwh * tarif_kwh +
    0.50;
```

Ligne	Variable	Valeur
L1	`duree_h`	
L2	`energie_kwh`	
L3	`energie_kwh_ent`	
L4	`energie_kwh`	
L5	`cout_euros`	

PROJET

Partie B — Décider : sécurité et mode de charge

Deuxième étape : la borne doit décider. Elle autorise ou refuse la charge selon les conditions d'exploitation, puis choisit le mode de charge adapté à la puissance demandée.

B1 — Lire une condition

La borne surveille la température de son module de puissance avec le code suivant.

Question 9

```c
float temperature_c = 58.5;

if (temperature_c < 65.0)
{
    printf("Module OK\n");
}
else
{
    printf("Surchauffe : charge coupee\n");
}
```

Qu'affiche ce fragment de code ?

- ◯ `Module OK` puis, sur la ligne suivante, `Surchauffe : charge coupee`
- ◯ Rien : aucune des deux branches n'est exécutée
- ◯ `Surchauffe : charge coupee`
- ◯ `Module OK`

B2 — Autoriser la charge

Question 10

La borne n'autorise une charge que si **toutes** les conditions d'exploitation sont réunies :

- tension réseau comprise entre 360 V et 440 V, **bornes incluses** ;
- courant demandé **au plus** égal à 63 A ;
- température du module de puissance **strictement** inférieure à 65 °C.

Écrire la condition manquante **directement dans le cadre pointillé**.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    float tension_v     = get_float("Tension reseau (V)   : ");
    float courant_a     = get_float("Courant demande (A)  : ");
    int   temperature_c = get_int("Temperature module (C) : ");

    if (                                                    )
    {
        printf("Charge autorisee\n");
    }
    else
    {
        printf("Charge refusee\n");
    }

    return 0;
}
```

Justifier brièvement le choix des opérateurs aux bornes de la plage de tension.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

SUJET

B3 — Surveiller la sécurité

Question 11

Un technicien a écrit le module de surveillance ci-dessous. Compilé sans option d'avertissement, il affiche parfois « Charge interrompue » alors qu'aucun défaut n'a été signalé.

```
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    float tension_v = get_float("Tension reseau (V)  : ");
    float courant_a = get_float("Courant demande (A) : ");
    int   mode      = get_int("Mode de charge (0/1/2) : ");
    int   defaut    = 0;

    if (360.0 <= tension_v <= 440.0)
    {
        printf("Tension hors plage\n");
        defaut = 1;
    }

    if (courant_a > 63.0 && mode = 2)
    {
        printf("Surintensite en mode ultra-rapide\n");
        defaut = 1;
    }

    if (defaut == 1);
    {
        printf("Charge interrompue\n");
    }

    return 0;
}
```

Repérer la ou les erreurs présentes dans ce programme en les **numérotant directement sur le listing** (1, 2…). Puis, dans le cadre ci-dessous, donner pour chaque numéro la **ligne corrigée** et le comportement fautif que la correction supprime.

PRO

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Partie C — Répéter : traiter une série de sessions

Troisième étape : la borne répète sans cesse des traitements. On étudie ici l'attente du verrouillage du câble, le cumul de l'énergie des sessions de la journée, la simulation d'une recharge à puissance dégressive, puis une montée en charge par paliers.

C1 — Attente du verrouillage

Question 12
Au branchement, la borne revérifie le verrouillage du câble toutes les 15 secondes tant que la durée d'attente écoulée ne dépasse pas 60 secondes.

```c
int temps_s   = 0;
int nb_essais = 0;

while (temps_s < 60)
{
    temps_s = temps_s + 15;
    nb_essais++;
}

printf("%d %d\n", nb_essais, temps_s);
```

Qu'affiche ce fragment ?

◯ 4 75 ◯ 4 60 ◯ 4 45 ◯ 5 60 ◯ 5 75 ◯ 3 45

PROJET

C2 — Cumul de l'énergie

Question 13

Pour le bilan d'un point de charge, un développeur inexpérimenté a écrit le programme suivant, qui cumule l'énergie délivrée par chacune des sessions de la journée.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int   nb_sessions    = get_int("Nombre de sessions : ");
    float energie_totale = 0.0;
    int   i              = 0;

    while (i < nb_sessions)
    {
        float energie_kwh = get_float("Energie de la session (kWh) : ");
        energie_totale = energie_totale + energie_kwh;
    }

    printf("Energie totale : %.1f kWh\n", energie_totale);

    return 0;
}
```

Ce programme ne se comporte pas comme prévu. Décrire le comportement observé à l'exécution, puis corriger le programme **directement sur le code**. Justifier ci-dessous.

PROJET

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

C3 — Recharge à puissance dégressive

Pour préserver la batterie, la borne réduit la puissance délivrée lorsque le niveau de charge augmente : le niveau gagne 20 points par pas de 10 minutes sous 60 %, puis 10 points de 60 à 80 %, puis 5 points au-delà. Le fragment ci-dessous simule cette recharge (`gain_pct` est le nombre de points de pourcentage gagnés pendant un pas de 10 minutes).

Question 14

Compléter le tableau en donnant, à la fin de **chaque tour de boucle**, les valeurs de `gain_pct`, `niveau_pct` et `minutes`. Numéroter vous-même les tours suivants et **s'arrêter au dernier tour effectivement exécuté** (les lignes en trop restent vides). ..... ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

```
int niveau_pct = 40;
int gain_pct   = 20;
int minutes    = 0;

while (niveau_pct < 90)
{
    if (niveau_pct >= 80)
    {
        gain_pct = 5;
    }
    else if (niveau_pct >= 60)
    {
        gain_pct = 10;
    }

    niveau_pct = niveau_pct + gain_pct;
    minutes    = minutes + 10;
}

printf("%d %d\n", minutes, niveau_pct);
```

Tour	gain_pct	niveau_pct	minutes
1			
2			

Question 15

Qu'affiche ce programme après exécution complète de la boucle ?

Note au correcteur : évaluer en cohérence avec le tableau rempli par l'étudiant à la question précédente (la bonne réponse est celle qui découle de sa *trace, même erronée), afin de ne pas pénaliser deux fois la même erreur.*

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PROJET

C4 — Montée en charge par paliers

La borne monte le niveau de charge par paliers de 7 points de pourcentage, en partant de 15 %. Le cahier des charges impose que :

- la boucle s'arrête dès que le niveau atteint **ou dépasse** le niveau cible saisi par l'utilisateur ;
- le niveau affiché ne dépasse **jamais** 100 %.

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int niveau_pct   = 15;
    int nb_paliers   = 0;
    int niveau_cible = get_int("Niveau de charge souhaite (%) : ");

    while (                                                    )
    {
        niveau_pct =                                                ;

        if (                                                    )
        {
            niveau_pct = 100;
        }

        nb_paliers++;
        printf("Palier %d : %d %%\n", nb_paliers, niveau_pct);
    }

    printf("Charge terminee en %d paliers.\n", nb_paliers);

    return 0;
}
```

Question 16
Compléter les trois éléments manquants **directement dans les cadres pointillés**.

. . . . . ◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Question 17
Indiquer ce que le programme affiche si l'utilisateur saisit un niveau cible de 40 %.

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

PRO

Partie D — Modulariser : des fonctions pour la borne

Quatrième étape : le logiciel grossit. On isole les traitements (comptage, facturation, décision d'engagement) dans des fonctions réutilisables et testables séparément.

D1 — Appeler une fonction existante

Le calcul du montant d'une session est confié à la fonction `cout_session`. On considère que cette fonction est **déjà écrite et implémentée dans le programme** : il n'est pas utile de connaître son contenu, seul son prototype compte :

```
float cout_session(float puissance_kw, float duree_h, float tarif_kwh);
```

Quatre appels à cette fonction apparaissent dans le programme :

```
/* Prototype disponible :
   float cout_session(float puissance_kw, float duree_h, float tarif_kwh); */

float c1 = cout_session(22.0, 1.5, 0.22);   /* appel 1 */
float c2 = cout_session(0.22, 22.0, 1.5);   /* appel 2 */
float c3 = cout_session(22.0, 1.5);         /* appel 3 */
float c4 = cout_session(22.0, 1.5, 0.22, 0.50); /* appel 4 */
```

Question 18 ♣
Cocher **tous** les appels acceptés par le compilateur.

◯ appel 1 ◯ appel 2 ◯ appel 3 ◯ appel 4

Question 19
Parmi les appels de la question précédent, indiquer lequel est la **bonne version** pour calculer le coût d'une session de 22 kW pendant 1,5 h à 0,22 €/kWh ?

◯ appel 1 ◯ appel 2 ◯ appel 3 ◯ appel 4

PROJET

D2 — Portée d'une variable

Un développeur tente de compter les sessions de la borne avec le programme suivant :

```c
void enregistrer_session(void)
{
    int compteur_total = 0;
    compteur_total++;
    printf("%d ", compteur_total);
}

int main(void)
{
    int compteur_total = 0;

    for (int i = 0; i < 3; i++)
    {
        enregistrer_session();
    }

    printf("| %d\n", compteur_total);

    return 0;
}
```

Question 20
Qu'affiche ce programme ?

◯ `1 1 1 | 0` ◯ `1 2 3 | 3` ◯ `1 1 1 | 3` ◯ `1 2 3 | 0`

Question 21
Réécrire la fonction `enregistrer_session` pour qu'elle **reçoive le compteur en paramètre** et **retourne sa nouvelle valeur** (sans variable globale), puis écrire la boucle de `main` qui l'utilise, de sorte que le programme affiche `1 2 3 | 3`.

PROJET

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

D3 — Écrire une fonction de décision

Question 22

On modularise maintenant la décision d'engagement de la charge.

On souhaite la fonction `int determiner_mode(float puissance_kw, int cable_verrouille)` qui retourne un **code entier** caractérisant la décision de la borne :

- si le câble n'est pas verrouillé (`cable_verrouille` vaut 0), retourner `-1` quelle que soit la puissance ;
- sinon, retourner `0` si la puissance demandée ne dépasse pas 22 kW, `1` si elle est comprise entre 22 kW (exclu) et 50 kW (inclus), `2` si elle dépasse 50 kW sans dépasser 150 kW ;
- si la puissance demandée dépasse 150 kW, retourner `3` (demande hors domaine de la borne).

La fonction ne doit rien afficher et doit retourner une valeur dans **tous** les cas.

Ecrire la définition (son corps) de cette fonction.

PROJET

◯0 ◯1 ◯2 ◯3 ◯4 ◯5

Partie E — Intégrer : le bilan de journée

Dernière étape : on assemble une fonction déjà écrite, une boucle et une structure de choix pour traiter les demandes de la journée et compter les sessions refusées.

Question 23

On dispose de la fonction `int determiner_mode(float puissance_kw, int cable_verrouille)`, que l'on considère **correcte**. Elle retourne `-1` si le câble n'est pas verrouillé, `0` en mode lent, `1` en mode rapide, `2` en mode ultra-rapide et `3` si la demande dépasse 150 kW. Sa définition est placée **après** le `main` ; son prototype est rappelé en tête du fichier.

Compléter les cadres pointillés pour que le programme traite les `nb_sessions` demandes de la journée, compte les sessions refusées (câble non verrouillé ou demande hors domaine) et affiche le mode des autres. Le choix selon le code retourné est réalisé avec une structure `switch`.

```c
#include <stdio.h>
#include <cs50.h>

int determiner_mode(float puissance_kw, int cable_verrouille);

int main(void)
{
    int nb_sessions = get_int("Nombre de sessions : ");
    int nb_refus    = 0;

    for (int i = 1; ..... ; i++)
    {
        float puissance_kw     = get_float("Puissance demandee (kW) : ");
        int   cable_verrouille = get_int("Cable verrouille (1/0) : ");

        int code = ..... ;

        switch ( ..... )
        {
            case ..... :
            case 3:
                nb_refus++;
                break;
            case ..... :
                printf("Session %d : mode lent\n", i);
                break;
            .....
                printf("Session %d : mode rapide ou ultra-rapide\n", i);
                break;
        }
    }

    printf("Sessions refusees : %d\n", nb_refus);

    return 0;
}

/* La definition de determiner_mode se trouve ici (supposee correcte). */
```

..... ◯0 ◯1 ◯2 ◯3 ◯4 ◯5